
What a Token-Keyed Residual Memory Can Carry

QingGo¹

Abstract

Sparse n-gram memory layers — DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash — inject a retrieved vector into the residual stream at every token, and are being scaled on the premise that the memory supplies knowledge the parameters do not hold. We show that the channel is bounded before any reader is chosen: the row identifier is a deterministic function of a short addressing window, so the data-processing inequality gives $I(\text{future}; e_t | h_t) \leq I(\text{future}; w_t | h_t)$. Depth, width, linearity and training budget do not appear. The window is not the two tokens an order-3 key suggests but twelve, because a kernel-4 dilation-3 convolution follows the gate; in V4.1-Flash, which removed the convolution, it is exactly four.

Testing that prediction, three readers differing only in the corpus behind the table — Wikipedia, code and STEM — write measurably different vectors (pairwise cosine 0.38 to 0.49 on 4B) and yet produce identical output distributions: total variation 0.0000 over 1500 items on 0.8B, and on 4B a largest cross-corpus spread of one item out of 600. Removing the answer-format SFT that collapses every arm to 2.5 generated tokens reveals the other half of the bound: all four format descriptors then move outside their split-half floors, and both pre-registered directional lexical predictions hold. Suppressing the chat scaffold is content-independent; the rest of the distribution carries the corpus’s surface statistics, which is the trigram prior the bound permits. Passage-conditioned recall never appears, with or without the ceiling.

Two measurements then point in different directions. A probe of the frozen table recovers about 59% of the trigram top-1 that explicit counts reach on the same corpus, and additional training makes it worse. Measured directly, the vector the reader injects has an effective dimensionality of 1.001 over 600 prompts, against 1.480 for the

hidden state it is gated on — the read-out is close to a constant function of its input. The design therefore fails for the bound’s reason and for a read-out defect at once, and only the first of those is unfixable at a twelve-token window.

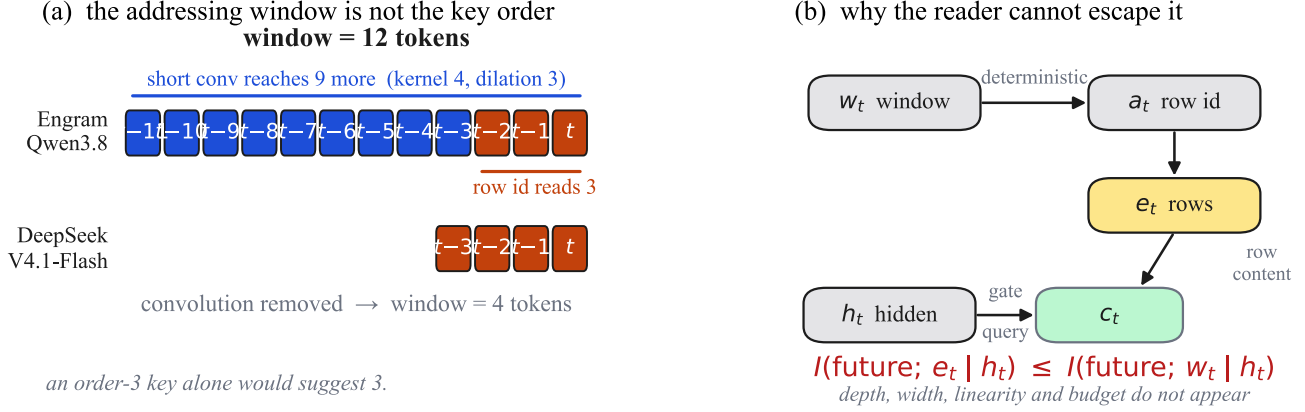
1. Introduction

Sparse n-gram memory is one of the few ways to add capacity to a language model without adding parameters. DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash all hash a short token window to a row of a large disk-backed table, project that row, and inject the result into the residual stream (X. Cheng et al., 2026). Memory Grafting scales pre-training with an offline conditional memory table (R. Cheng et al., 2026), and XMemTransfer adapts a table across model families with a target-side reader (OLAResearch, 2026). The premise is that the table holds associations the weights compress away, and that a large enough table behind a good enough reader will supply them.

This paper asks what such a channel can carry, and answers it in a way that does not depend on the reader. The argument is short. The row identifier is a deterministic function of a short addressing window w_t ; the reader’s contribution is a deterministic function of that row and the hidden state h_t . The data-processing inequality then bounds the memory’s contribution to the future by the window’s. Reader depth, width, linearity and training budget do not appear because they cannot. The bound is elementary; what is not elementary is that it was not written down for a family that three production systems are scaling, and that the window it depends on is not the one the headline key order suggests.

That last point is where the bound becomes useful rather than obvious. An order-3 key reads two tokens, but the Engram and Qwen3.8 readers follow the gate with a depthwise convolution of kernel 4 and dilation 3, so the contribution at a position draws on memory rows up to nine positions back, each of which reads two tokens further: the true window is twelve tokens, not two. DeepSeek-V4.1-Flash removed that convolution, so its window is exactly four. The family

¹Independent Researcher. Correspondence to: QingGo <zyqingjohn@qq.com>.



an order-3 key alone would suggest 3.

Figure 1. The channel is bounded by its addressing window, and the window is larger than the key order suggests. (a) An order-3 key reads two tokens, but the Engram and Qwen3.8 readers follow the gate with a kernel-4 dilation-3 depthwise convolution, so the contribution at a position draws on rows nine positions back, each of which reads two tokens further: twelve tokens, not two. V4.1-Flash removed the convolution and its window is four. (b) Because e_t is a deterministic function of w_t , no reader can exceed what the window already determines and h_t has not retained.

spans a factor of three in the one quantity that bounds it, and that quantity is visible only by reading the convolution parameters.

The bound makes a sharp and testable prediction about content, and we test it on two backbones. Train three readers that differ only in the corpus feeding the table — Wikipedia, code and STEM text — and graft each into the same frozen model at the same layer under the same prompts. If the memory carried corpus content, the three should answer differently. Under the standard recipe they do not: on 0.8B over 1500 items per arm the answer-label distribution, the leading-surface distribution and their joint differ by a total-variation distance of exactly 0.0000, and on 4B the largest cross-corpus spread is a single item out of 600. This is not a failed manipulation. The three readers write visibly different vectors at the answer position, and each is best on its own corpus in a cross-evaluation; the corpus changes the part of the trigram statistic the weights already encode, and under this recipe that part leaves no trace.

It would be a mistake to stop there, and we did not. Every arm in that experiment collapses to a two-token answer, so “no format difference” could mean “no room for one”. Removing the answer-format supervision that causes the collapse changes the conclusion, and the change is the bound’s other half rather than a counterexample to it: all four format descriptors move outside their split-half floors and both pre-registered directional lexical predictions hold, with the Wikipedia reader emitting far more prose markers than the code reader. The layering is exactly what the bound predicts. Suppression of the chat scaffold is content-independent — all three readers suppress it and differ from one another by at most 0.01 — while the rest of the distribution carries the corpus’s surface statistics. That dependence is

the trigram prior the channel is permitted to carry. What never appears, with or without the ceiling, is passage-conditioned recall; the knowledge probes that require the passage return zero.

Finally we ask how much of the permitted prior actually survives the reader, and the answer complicates the story in a way we did not expect when we began. Probing the frozen table directly shows that it does hold a recoverable trigram prior, well above the majority floor and destroyed by shuffling, but that the read-out recovers only about 59% of what explicit counts reach. Measuring the injected vector directly shows why: over 600 diverse prompts it spans effectively one dimension, against 1.480 for the hidden state it is gated on. So the design fails for two independent reasons — one the bound, which is not fixable at a twelve-token window, and one a read-out that has collapsed, which is. Separating them matters, because it changes what the negative result is evidence for, and we are explicit below about the limits of our ability to do so.

Our contributions are these.

1. We state and prove a bound on what a token-keyed residual memory can carry, in a conditional form that accounts for the gate’s dependence on the hidden state, and we identify the addressing window of each production design in the family — twelve tokens for Engram and Qwen3.8-Flash-Next, four for V4.1-Flash.
2. We test its content prediction directly on two frozen backbones with three corpus-matched readers, and show that the three inject measurably different vectors while producing identical output distributions.

3. We show that the null is scoped rather than absolute: removing the answer-length ceiling reveals the corpus’s surface prior, exactly where the bound says it should be, while passage-conditioned recall remains at zero.
4. We measure the ceiling from the other side, probing the frozen table directly and finding that the read-out recovers about 59% of the trigram top-1 explicit counts reach, a loss that does not improve with more training.
5. We measure the injected vector itself and find that it is nearly a constant function of its input, which means the null has two candidate causes rather than one; we report what our experiments do and do not separate.
6. We falsify the obvious repair. At matched storage, longer keys do not pay on any corpus or budget tested, so the short window is not the constraint that binds.
7. We report the audit that made these results checkable, including ten occasions on which a claim stood because the check that would have falsified it did not exist or silently skipped, and one pre-registered prediction that failed.

2. Related Work

2.1. External Memory Layers

DeepSeek Engram (X. Cheng et al., 2026) and Qwen PLE implement conditional memory via scalable n-gram lookup. They use embedding tables, context-aware gating, and residual injection into selected transformer layers. Memory Grafting (R. Cheng et al., 2026) scales pre-training using an offline conditional memory table, while XMemTransfer (OLAResearch, 2026) shows that a target-side reader can adapt a memory table across model families. Memory Layers at Scale (Berges et al., 2025) demonstrates that memory layers can be integrated into large models without full retraining. Product-key memory layers (Lample et al., 2019) provide a related sparse lookup mechanism, and recent latent n-gram architectures such as Lngam v2 (Zheng et al., 2026a; 2026b) and Tensorizing Engram (Zhou et al., 2026) improve the parameter efficiency and interpretability of discrete memory addressing. In the small-model regime, PEMA (Kim et al., 2023), memory-augmented training (Zhong et al., 2022), Knowledge-in-Context (Pan et al., 2022), and plug-and-play knowledge injection (Zhang et al., 2023) show that external memory can be adapted after pretraining without full retraining.

2.2. Long-Term and Agent Memory

A parallel line of work treats memory as a first-class agent resource. MemGPT (Packer et al., 2023) introduces operating-system-style memory paging for LLM agents;

HippoRAG (Gutiérrez et al., 2024) and From RAG to Memory (Gutiérrez et al., 2025) convert retrieval corpora into long-term memory structures; Zep (Rasmussen et al., 2025), Memori (Borro et al., 2026), and MemOrb (Huang et al., 2025) provide persistent or plug-and-play memory layers for conversational and agentic settings. These systems target long-range semantic memory, whereas our work focuses on a narrower, auditable token-level n-gram memory that is useful in low-entropy local continuations.

2.3. Non-Parametric Language Models

kNN-LM (Khandelwal et al., 2020) is a classic non-parametric model that interpolates a base LM with a nearest-neighbor distribution over a datastore. Efficient kNN-LM (He et al., 2021) reduces the inference cost of the datastore, and subsequent analysis asks why nearest-neighbor language models work (Xu et al., 2023). Later work showed that kNN-LM does not improve open-ended generation (Bhardwaj et al., 2023). NGM (PioneerQyw, 2026) provides a training-free n-gram memory hook. Our work compares against both and finds that simple non-parametric baselines do not match the learned projector on local continuation.

2.4. Distribution-Level Memory

MemSFT (Group, 2026) and TokenMem (T. Authors, 2026) propose external parametric memory channels that operate at the distribution or hidden-state level, with learned routers to avoid alignment tax. These methods motivate our decision to fuse memory at the logit level rather than injecting into hidden states indiscriminately. Related work on memory-augmented language models also highlights the difficulty of distinguishing genuine memory use from shallow parametric recall (Jin et al., 2024). Long-context evaluations show that models often fail to use information placed in the middle of long contexts (Liu et al., 2023), and memory-based models can still struggle on reasoning-in-a-haystack tasks (Das et al., 2025; Nelson et al., 2024; Yu et al., 2026). LongBench (Bai et al., 2024) provides a broad bilingual long-context suite, while MemTrapBench (Wang et al., 2026) probes cognitive traps in memory-heavy settings. Earlier experiments in our project found that hidden-state injection without careful orthogonalization and gating can produce large real-vs-control gaps that are not attributable to the memory content.

2.5. Retrieval Reliability and Calibration

Adaptive retrieval methods such as Self-RAG (Asai et al., 2023) and FAIR-RAG (Asl et al., 2025) decide when retrieval is necessary; reliability-aware RAG systems further estimate whether retrieved evidence should be trusted (Hwang et al., 2024; Jiang et al., 2026; Shin et al., 2026;

Xu et al., 2026), and RAGRouter-Bench (R. Authors, 2026) benchmarks learned query-routing policies. Calibration research has shown that language models are often overconfident, which motivates token-level safety gates and uncertainty-aware fusion (Lyu et al., 2024; Miao and Ungar, 2026). On code tasks, localized calibrated uncertainty helps identify unreliable predictions (Gros and Devanbu, 2025). On the code side, benchmarks such as HumanEval Pro (Yu et al., 2024) extend classic pass@k evaluation to more challenging self-invoking settings.

2.6. Parameter-Efficient Adapters

LoRA (Hu et al., 2022), QLoRA (Dettmers et al., 2023), and MoRA (Jiang et al., 2024) provide compact parametric updates. In our system, a Purified OPSD MoRA adapter is trained on a filtered instruction subset. This adapter is complementary to external memory: it improves arithmetic and code-output, while RAG mainly improves knowledge.

3. What a Token-Keyed Residual Memory Can Carry

3.1. N-Gram Addressable Memory

Let a context be a token sequence $c = (c_1, \dots, c_t)$. We maintain sparse counts for each n-gram of order n :

$$p_{m(y|c)} = \frac{\text{count}(c, y)}{\sum_y \text{count}(c, y)}. \quad (1)$$

The memory also returns the longest matched order and an external value index that can be audited. This memory is non-parametric, transparent, and can be rebuilt from any corpus.

3.2. The Bound

The Engram / PLE line injects a retrieved vector into the residual stream. It is worth stating precisely what such a channel can carry, because the answer does not depend on the reader.

Let w_t denote the addressing window ending at position t , and let the row identifier be a deterministic function of it, $a_t = A(w_t)$, so that the retrieved memory is $e_t = E(a_t)$. The contribution to the residual stream is $c_t = F(h_t, e_{\{t-r..t\}})$, where the backbone hidden state h_t enters through the gate as the query and r is the reach of the short causal convolution that follows the gate. In the Engram and Qwen3.8 designs the row id reads the last $n - 1$ tokens and the convolution has kernel 4 with dilation 3, so the convolution at a position reads memory rows up to nine positions back, each row itself reading two tokens further, and the window reaches eleven tokens back — twelve tokens in all, not the two that $n = 3$ alone would suggest. Because e_t is

a deterministic function of the window, the data-processing inequality gives

$$I(\text{future}; e_t | h_t) \leq I(\text{future}; w_t | h_t). \quad (2)$$

Three consequences follow.

First, the bound is independent of the reader. Depth, width, linearity and training budget do not appear in it, so no reader, however expressive, can exceed it. This turns our earlier failures with hidden-state readers and direct residual injection from an empirical observation into a necessary one.

Second, it constrains addressing rather than capacity. The memory can only supply what the window already determines and the hidden state has not retained, so no context-conditioned recall is possible when the determining information lies outside the window. For a question answered from a passage, the window at the answer position carries the prompt’s format, not its content; the memory may still sharpen **how** the answer is emitted, which is the format effect we measure, but not **which** answer is correct.

Third, it does not say the memory is useless. Backbone weights are a lossy compression of the training corpus, and rare n-gram statistics are among what is compressed away. The bound leaves room for precisely that, and it predicts where: gains should concentrate on rare n-grams and on continuations the window determines, and should vanish elsewhere. Two parts of that prediction we can test and do: knowledge probes that require the passage return zero, and the measurable residual is a format prior rather than content. The rarity half we cannot test, and it is worth saying why. Correlating gain with context rarity requires a count for the window n-gram, and no affordable reference corpus supplies one: over a 3.1M-token corpus, 93% of the evaluated windows have a zero count at the table’s own addressing order of three, so the tertile split degenerates, and at the window scale itself essentially every twelve-gram is unique in any corpus one could count. The rarity prediction therefore remains a prediction.

3.3. Scope: The Family, Not One Implementation

The scope is the family, not one implementation. DeepSeek Engram and Qwen3.8-Flash-Next address with 2- and 3-grams and keep the convolution, giving a window of twelve tokens; DeepSeek-V4.1-Flash uses 2-, 3- and 4-grams but removed the convolution, so its window is exactly four tokens and is in that respect the tightest of the three. Enlarging the table does not relax the bound, and neither does making the address depend on more of the same window.

One clarification is worth making, because it is easy to over-read. The bound is conditional on h_t , so an address

that depends on the hidden state as well as the window does not escape it: if $a_t = A(w_t, h_t)$ then e_t is still a function of (w_t, h_t) given h_t , and $I(\text{future}; e_t | h_t) \leq I(\text{future}; w_t | h_t)$ is unchanged. What escapes the bound is addressing on information that h_t has **not** retained — retrieval over the full context, which is a lossy summary’s complement rather than a wider view of the same neighbourhood.

4. Does Memory Content Reach the Output?

We tested the bound’s content prediction directly on two backbones. We train three readers that differ only in the corpus feeding the table — Wikipedia, code and STEM text — and graft each into the same backbone at the same layer under identical prompts. If the memory carried corpus content, the three should answer differently.

4.1. Under the Standard Recipe, They Do Not

On Qwen3.5-0.8B in fp32, over 1500 items per arm, the answer-label distribution, the leading-surface distribution and their joint all differ by a total-variation distance of exactly 0.0000, and every pre-registered directional lexical prediction fails, with zero discordant pairs for code markers. On Qwen3.5-4B in bf16 — the backbone on which the format effect was first observed — the same null holds over 600 items per arm: the largest cross-corpus spread is a single item, below the split-half floor of the arm that produced it. What the corpus changes is the part of the trigram statistic the backbone weights already encode, and under this recipe that part leaves no trace in the output.

4.2. The Manipulation Reached the Injection Point

That the null is not a failed manipulation is established independently of the output. On 0.8B the three readers write measurably different vectors at the answer position (pair-wise cosine similarity 0.73 to 0.76, relative L_2 distance 0.69 to 0.78, norms within a factor of 1.10); on 4B the separation is larger, cosine 0.38 to 0.49. In a 3×3 cross-evaluation each reader is best on its own corpus. The frozen tensors are bit-identical across arms and the adapter-norm ratios span only 0.886 to 1.045, well inside the threefold bound that would make the arms incomparable.

4.3. The Effect Is Large, and Larger on the Bigger Backbone

The contrast with the zero-injection arm shows the channel is not merely inert, and the effect is far larger on the 4B backbone. There, zero injection makes the model emit chat scaffolding on 95.8% of items (mean 31.2 generated tokens), while injecting a reader trained on any of the three corpora suppresses it to 0.0% (mean 2.6 tokens), with not one of the 600 items sharing its first token with the zero-in-

jection arm ($p = 1.6 \times 10^{-173}$, McNemar exact). On 0.8B the same shift moves scaffolding from 19.5% to 0.0% and changes the first generated token on all 1500 items ($p = 1.3 \times 10^{-88}$); the reader-disabled and no-reader arms are bit-identical, so the noise floor is exactly zero rather than merely small. The graft therefore does something large and reproducible — it pushes the model off the chat template — but that shift does not depend on what the table contains.

4.4. Removing the Ceiling

This null has a scope, and testing it changes what the result means. On both backbones the injected arms saturate at 2.5 to 2.7 generated tokens, so “no format difference” could mean “no room for one”. We therefore ran the regime that removes the saturation — no answer-format SFT, where generated length returns to 11 to 21 tokens — over all six arms on 600 items, so the same pre-registered rule applies without adjustment. The reader-disabled and no-reader arms are again bit-identical, giving a noise floor of exactly zero. The two halves of the rule then disagree, and the disagreement is the result. The coarse label taxonomy still reports a perturbation artifact, because the scaffold rate spans only 0.0100 against a threshold of 0.0122; every finer format descriptor reports content dependence. That division is why the co-primary descriptors were pre-registered alongside the taxonomy, on the stated grounds that the taxonomy can be insensitive to a real format shift. All four move outside each arm’s own split-half sampling floor (leading-surface total variation 0.1433 against a floor of 0.0237; first-token 0.4983 against 0.2176), and both pre-registered directional lexical predictions hold: the code reader emits more code markers ($+0.0333$, one-sided $p = 3.9 \times 10^{-4}$) and the Wikipedia reader emits far more prose markers ($+0.6933$, $p = 4.5 \times 10^{-34}$), where under saturation both predictions had failed outright.

4.5. The Layering Is What the Bound Predicts

So the correct statement is layered rather than absolute, and it is the layering the bound predicts. Suppressing the chat scaffold is content-independent — all three readers suppress it, and they differ from each other by at most 0.01. The rest of the output distribution is content-dependent, and that dependence **is** the trigram prior the bound allows the channel to carry: a reader trained on Wikipedia supplies Wikipedia-like surface statistics, one trained on code supplies code-like ones. What never appears, with or without the ceiling, is passage-conditioned recall — the knowledge probes that require the passage return zero. The channel carries the corpus prior and nothing above it.

4.6. How Much Prior Is Actually There

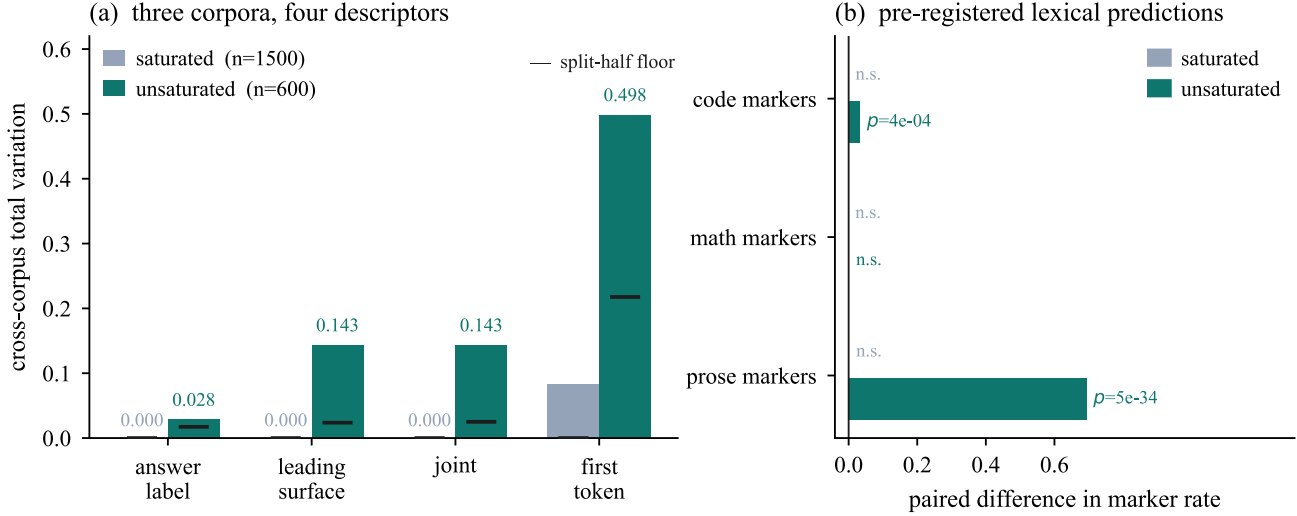


Figure 2. Removing the answer-length ceiling reveals content dependence that the saturated regime could not show. (a) Cross-corpus total variation for four format descriptors, against each arm’s own split-half floor (short ticks). Under the standard recipe three descriptors are pinned at exactly zero; with the ceiling removed all four move outside their floors. (b) The pre-registered directional lexical predictions. Both were refuted under saturation and both hold without it. Math markers remain non-significant in both regimes, which is the expected result for a contrast the corpus does not separate.

The bound permits the memory to carry a corpus’s rare n -gram statistics, so the useful question is how much of that prior the frozen table actually holds and how much of it the read-out recovers. We probe the raw 2560-dimensional rows directly, on held-out windows, against explicit counts on the same corpus as a reference frame. Majority-class prediction gives a floor of 0.0537 top-1. A count bigram reaches 0.2264 and a count trigram 0.2535. A ridge probe on the frozen rows reaches 0.1280; an MLP reaches 0.1502 at its best epoch and 0.1225 at 24 epochs, so additional training makes it worse rather than better. Shuffled-row controls collapse below the majority floor at 0.0459 and 0.0373. The table therefore does hold a recoverable trigram prior — well above the floor and destroyed by shuffling — but the read-out recovers only about 59% of what explicit counts reach.

4.7. The Read-Out Does Not Transmit It

That gap is not a small correction, and measuring the injected vector directly shows how large it is. Over 600 diverse prompts the vector the reader adds to the residual stream has an effective dimensionality of 1.001: 99.93% of its variance lies in a single direction, and two prompts are separated by about two degrees. The hidden state the reader is gated on sits at 1.480, with 81.9% in its leading direction. A generic linear map does not do this — three random maps of the same shape leave the hidden state’s ratio at 1.454, 1.472 and 1.485 — and replacing the retrieved rows with permuted ones, at the same checkpoint, moves the injected direction by only about five degrees.

The consequence is that the content-independence we measure under the standard recipe has two candidate explanations, and our experiments separate their consequences without separating the explanations themselves: either the window has nothing to give beyond the hidden state, or the read-out does not transmit what the window has. The bound is a theorem and holds in both cases, but the second explanation is a defect rather than a limit. The table probe is the reason we do not simply attribute the whole null to the bound: an explicit count model over the same trigram reaches 0.2535 top-1, which is above the majority floor and therefore genuine information that a better read-out could in principle transmit.

4.8. The Obvious Repair Does Not Pay

Nor does lengthening the key. Replacing the 4-gram address with 8-token, 16-token and longest-suffix memories at matched storage ties or loses at every budget and training size on both corpora, so the short window costs nothing measurable. That the window can be widened without gain, while the corpus the table is built from cannot be changed without gain either, is the same statement twice: the channel’s output is fixed by what the window already determines.

5. The Logit-Correction Interface Under the Same Bound

The remainder of the paper concerns a second interface to the same resource. Instead of injecting the retrieved row into the residual stream, a small projector adds a per-

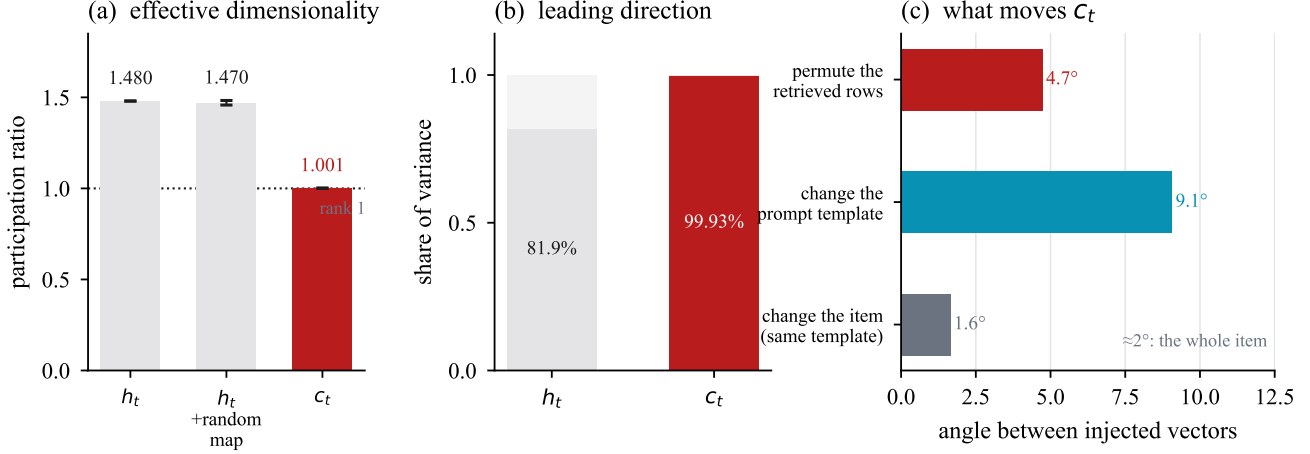


Figure 3. The read-out is close to a constant function of its input. (a) Effective dimensionality (participation ratio) of the injected vector, of the hidden state it is gated on, and of that hidden state after a random linear map of the same shape. The random maps are the control: a generic map does not collapse the hidden state, so the collapse to 1.001 is the read-out’s. (b) Share of variance in the leading direction. (c) What actually moves the injected vector. Changing the prompt template moves it further than permuting the retrieved rows, and changing the item within a template moves it least of all.

token correction to the logits, $\log p_{\text{fused}}(y) = \log p_{b(y)} + \alpha_t \log p_{m(y)} + \beta_t$, with a learned token policy deciding when the correction is active. We describe it here because reading it under the bound changes what its results are evidence for, and we report it in that light rather than as a separate contribution.

The distinction between the two interfaces is real but it is not an exemption. Logit-level fusion of an n-gram memory is **also** a function of the addressing window, so the bound applies exactly as it does to the residual graft. What the projector gets is a better interface, not a larger allowance: the permitted prior is the same in both cases. The graft loses it twice, once through the read-out and again through a residual-stream bottleneck in which a single vector must be disentangled by the backbone before it can affect any logit; the logit correction loses it once, and spends it in the basis where it is used.

5.1. Method

The projector is a small MLP $f_{\theta(h_t, m_t)} = (\alpha_t, \beta_t)$ over the last hidden state and a feature vector m_t carrying the matched n-gram order, base and memory entropy, the density ratio, both top-1 probabilities, memory/base agreement, and a task one-hot. Its final layer is zero-initialized so the projection begins as the identity. A logistic policy on the same features predicts whether fusion will help at the current token and disables it otherwise, which is what keeps open-ended generation from degrading. Per-task calibration supplies a scale, bias and temperature, and only projector and policy parameters are trained; the backbone and the memory table stay frozen.

Table 1. Joint system mean answer log-probability, three seeds; higher is better. RAG carries knowledge, the adapter carries arithmetic and code-output, and PLE alone carries neither. Standard deviations across seeds are below 0.012 in every cell.

System	Knowledge	Arithmetic	Code-output
Base	−8.140	−7.365	−14.250
RAG	−6.748	−7.365	−14.250
PLE	−8.140	−7.492	−14.250
MoRA	−7.691	−7.114	−12.292
RAG+MoRA	−6.704	−7.114	−12.292
All	−6.704	−7.237	−12.292

5.2. What It Buys

The gain is confined to low-entropy local continuation, which is the region the bound leaves open, and it scales with projector training data. On code continuation over 100 samples with five seeds the projector beats fixed calibration by +0.1044 nats (bootstrap 95% CI [0.0251, 0.1866]); at 10k samples the margin grows to +0.2249 with CI [0.1436, 0.3255] and all five seeds are positive. Number-like continuations are more sensitive to calibration and often want a near-zero scale.

The joint system separates by resource rather than by strength. RAG supplies document-level evidence and is the only component that moves the knowledge column; the parametric adapter moves arithmetic and code-output; PLE alone moves nothing, and the combination is worth roughly its best member. On HumanEval the same picture holds from the other direction: greedy pass@1 is 0.22 for the base

model and 0.10 for BM25+PLE, the two pass sets overlap on a single problem out of 50, and under sampling over 10 problems at temperature 0.8, top- p 0.9 the ordering reverses (0.40 to 0.70). Neither direction is a reliable gain, and we read both as the same statement: a token-level n -gram prior sharpens the base model’s local mode rather than adding capability.

Knowledge QA confirms the boundary. On 200 TriviaQA RC validation examples the frozen 0.8B model reaches exact match 0.005, and PLE does not move it, because the required knowledge is not in local n -gram continuations. Training-free baselines behave the same way: NGM leaves the distribution essentially unchanged, and kNN-LM raises hit@1 from 0.505 to 0.540 while worsening NLL from 2.633 to 2.847 on the same evaluation set. An external LLM judge agrees with the pass-based metrics (mean score 0.60 for base against 0.20 for BM25+PLE on HumanEval 50, with a parallel API rerun differing by up to 0.20, so we treat it as secondary evidence). Full per-condition tables, the sensitivity sweep over n -gram order and memory-bank size, the algorithm listings, and the system figures are in the appendix.

6. Analysis

6.1. When Does the Channel Help

It helps when the true next token is highly predictable from local n -grams: code, identifiers, repeated structural patterns, name-like continuations. In those cases the memory distribution has low entropy and the correction can sharpen the model without adding noise. This is exactly the region the bound leaves open.

6.2. When Does It Fail

It fails when the answer depends on world knowledge or long-range reasoning. The memory does not contain semantic knowledge, and adding it to the logits can produce confident but wrong continuations, which is why TriviaQA sits at 0.005 and why unconditional fusion degrades open-ended generation without the token policy. For token-keyed memory this failure is not contingent but necessary: the bound shows the channel cannot carry what the addressing window does not already determine.

6.3. The Bound Does Not Partition the Mechanisms the Way One Might Expect

Two mechanisms escape the bound, and only two: retrieval, which addresses by query rather than by a fixed token window, and parametric adapters, which are not a memory channel at all. PLE, RAG and adapters are therefore not three points on one axis of strength but three different

interfaces to three different resources, and only the first of them is bounded in the way this paper describes. That reframing is testable against our own numbers and it holds: the projector’s gain is confined to low-entropy local continuation and is absent on knowledge tasks, where the determining information lies outside the window.

6.4. Why a Learned Projector Rather Than Fixed Calibration

Fixed calibration selects one scale and bias for all tokens. A learned projector varies the trust it places in memory by context, using a large positive scale on code and a near-zero scale on ambiguous open-ended text. That is what the improvement over fixed calibration measures, and it is a statement about the interface rather than about the memory’s content, which is bounded in both cases.

7. A Note on Method

Two lessons from this project bear on how the results should be read.

The first is that a channel must be checked against its information-theoretic ceiling before its capacity is scaled, and a memory module must be evaluated by whether it uses memory content rather than by whether loss improves. Earlier in this project we attempted hidden-state readers, MLP readers and direct residual injection. These approaches often improved loss-like metrics but failed the real-versus-control test, and the bound explains why they had to: a reader translating a token-keyed row into the residual stream cannot carry context the window does not determine, so the real and control arms converge once the reader is well trained.

The second is less comfortable. Ten times in this project a change was made, a claim was written, and the check that would have falsified the claim either did not exist or silently skipped, so the claim stood unexamined. A runtime used a different row-identifier implementation than the one that had been proved correct. A convolution believed absent was in the executed path. A loop body was indented such that a metric scored four items instead of 1500. Four golden tests were skipping rather than passing on the machine that produced every number. And the forward pass of the reader underlying every graft measurement had never been compared to the official mathematics it claimed to reuse, which we finally did: they agree bit-for-bit, which is the outcome that makes the rest of this paper readable rather than a cautionary tale. None of these were found by re-reading code. Each was found by making the failure detectable — invariant assertions, regression tests that fail on the specific bug, a flag that converts a silent skip into a failure, and manifests recording a fingerprint of what was actually measured. In every case the durable fix was the detection mechanism, not

the correction. A further two mistakes of the same family, a constant copied from the wrong fixture and a skip flag applied to an asset that is not in the repository, were caught within minutes once the checks were real; the difference between the two families is the whole lesson.

A third observation is about failure that is not a mistake. Our final experiment pre-registered a prediction — that the injected vector would be more nearly constant for a task whose addressing window is mostly a fixed instruction suffix than for tasks whose windows carry query text — and the prediction was wrong. Within-task similarity came out at 0.9994 and 0.9992, a difference of +0.0002 that a permutation test over task labels declined to distinguish from noise ($p = 0.27$). What fixing the statistic and its refutation condition in advance bought us was not protection from a wrong result. It is that the wrong result is legible: because the rule could not be adjusted afterwards, the failure pointed straight at the near-constant read-out, which is a stronger finding than the prediction would have been. A full account of the audit, including the complete list of unrun checks, is in the project repository.

8. Limitations

- HumanEval is limited to 50 problems and pass@k to 10 problems with 3 samples, so the code results are directional rather than benchmark-scale.
- TriviaQA exact match is very close to zero (0.005), so the paper reports a boundary rather than a positive result. LLM-judge scores show run-to-run variance and are secondary evidence.
- The unsaturated regime that reveals the corpus’s surface prior is measured over 600 items on one backbone under a different training recipe; it is a secondary regime and is reported as one. The saturated null it qualifies is measured on two backbones.
- The content-independence we measure under the standard recipe is consistent with two mechanisms — an addressing window that carries nothing beyond the hidden state, and a read-out that transmits almost nothing — and we do not fully separate them. Direct measurement shows the second is real (effective dimensionality 1.001; a five-degree response to permuted rows), and the table probe shows the window’s trigram statistics sit above the majority floor, so the first is not the whole story. The bound holds either way, but the share of the design’s failure attributable to it rather than to the read-out is left open, and the random-linear-map control we use to argue the collapse is the read-out’s was chosen after seeing the result rather than pre-registered.
- Two of the five per-seed artifacts behind the 10k projector result were not retained in the released bundle. The three that were reproduce the published table exactly, and the per-seed figure marks which bars come from which source; the remaining two rows are reported from the original run and are not independently reproducible from the release.
- The bound’s rarity prediction — that gains concentrate on rare n-grams — is stated but not tested, and we show above that it is not testable by corpus counting at the window scale with the resources available to us.
- The reader-fidelity check covers the read-out, not the addressing: we verify that our reader reproduces the official mathematics bit-for-bit at production geometry, but the table itself is accessed through our own row-identifier implementation, which is separately pinned against two independent implementations.
- The official Qwen3.8 model attaches its memory layer at 0-based layer 1, while our grafts inject at layer 2. The bound is a property of the reader’s input-output map and is layer-independent, so the results above are unaffected, but the layer-1 configuration is untested rather than refuted.
- Co-training the table with the backbone — the configuration all three production systems use, and the one most likely to change the value of the channel within the bound — is not tested here, at this scale.
- The bound is elementary — a data-processing inequality — and we do not claim otherwise. Its value is in the window arithmetic it forces on a family that three production systems scaled without writing it down, and in the experiment that tests its content prediction.
- Public projector and adapter weights are available on Hugging Face, but the upstream Qwen model weights themselves are not redistributed. CPU deployment throughput is about 2 tok/s with the current unoptimized eager implementation.

9. Conclusion

We asked what a token-keyed residual memory can carry, and the answer is set before any reader is chosen. The row identifier is a deterministic function of a short addressing window, so the memory’s contribution to the future cannot exceed what that window already determines and the hidden state has not retained. In the Engram and Qwen3.8-Flash-Next designs that window is twelve tokens rather than the two an order-3 key suggests, because a kernel-4 dilation-3 convolution follows the gate; in DeepSeek-V4.1-Flash the convolution was removed and the window is exactly four.

The prediction this makes about content survives a direct test. Three readers that differ only in the corpus feeding the table write measurably different vectors and, under the standard recipe, produce output distributions that are identical to within a single item out of 600. Removing the answer-length ceiling does not break the bound but completes it: the corpus’s surface statistics then appear in the output, which is the trigram prior the channel is permitted to carry, while passage-conditioned recall remains at zero. The channel is a channel for a corpus’s surface prior and not for knowledge, and which of the two it is, is fixed by the addressing window.

The measurements that size what remains point in different directions. The read-out recovers about 59% of the trigram top-1 that explicit counts reach, and more training makes it worse, so the loss is architectural rather than a matter of budget; measured directly, the vector it injects is nearly a constant, with an effective dimensionality of 1.001 against 1.480 for the hidden state it reads. Because the table’s trigram statistics sit well above the majority floor, this gap is a target rather than a verdict: the design fails for the bound’s reason and for a read-out defect at the same time, and only the first of those is unfixable at a twelve-token window. Lengthening the key does not pay at matched storage on any corpus or budget we tested. What is left is therefore the complement — retrieval, which addresses by query, and parametric adapters, which are not a memory channel at all — together with the read-out itself.

10. Acknowledgement

We thank the open-source community for the PLE/Engram, Qwen, and related memory infrastructure used in this study. This work was carried out as an independent low-resource research project.

11. Data Availability

All source code, evaluation scripts, container definitions, evaluation cards, and checksums are publicly available in the project repository and in the Hugging Face artifact repository. The released artifact bundle includes PLE projector checkpoints, Purified OPSD MoRA adapters, projector datasets, configurations, and the raw evaluation result files, including the per-item injected vectors behind the read-out measurement. The PLE memory tables can be rebuilt from the published corpus construction scripts, and upstream Qwen model weights are not redistributed.

References

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. *Self-RAG: Learning to Retrieve, Generate, and Cri-*

tique through Self-Reflection, 2023. <https://arxiv.org/abs/2310.11511v1>

Asl, M. A., Asgari-Bidhendi, M., and Minaei-Bidgoli, B. *FAIR-RAG: Faithful Adaptive Iterative Refinement for Retrieval-Augmented Generation*, 2025. <https://arxiv.org/abs/2510.22344v1>

Authors, R. *RAGRouter-Bench: Benchmarks for Learned Query Routing in Retrieval-Augmented Generation*, 2026. <https://arxiv.labs.arxiv.org/html/2602.00296>

Authors, T. *TokenMem: Faithful Knowledge Injection for Frozen LLMs*, 2026. <https://arxiv.org/html/2607.22625>

Bai, Y., Lv, X., Zhang, J., and others. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*, 2024. <https://arxiv.org/abs/2308.14508>

Berges, V.-P., Oguz, B., Haziza, D., and others. *Memory Layers at Scale. International Conference on Machine Learning*, 2025. <https://mlanthology.org/icml/2025/berges2025icml-memory/>

Bhardwaj, R., Vaddamanu, S., and others. *kNN-LM Does Not Improve Open-ended Text Generation. Proceedings of EMNLP*, 2023. <https://aclanthology.org/2023.emnlp-main.929/>

Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. *Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents*, 2026. <https://arxiv.org/abs/2603.19935v1>

Cheng, R. et al. *Memory Grafting: Scaling Language Model Pre-training via Offline Conditional Memory*, 2026. <https://papers.cool/arxiv/2605.20948>

Cheng, X. et al. *Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models*, 2026. <https://github.com/deepseek-ai/Engram>

Das, P., Ko, C.-Y., Dai, S., Kollias, G., Chaudhury, S., and Lozano, A. *Can Memory-Augmented Language Models Generalize on Reasoning-in-a-Haystack Tasks?*, 2025. <https://arxiv.org/abs/2503.07903v1>

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. *QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems*, 2023.

Gros, D., and Devanbu, P. *Localized Calibrated Uncertainty in Code Language Models*, 2025. <https://arxiv.org/abs/2512.24560v1>

Group, L. *MemSFT: Mitigating Alignment Tax with an External Parametric Memory*, 2026. <https://github.com/LUMIA-Group/MemSFT>

- Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*, 2024. <https://arxiv.org/abs/2405.14831v3>
- Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., and Su, Y. *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models*, 2025. <https://arxiv.org/abs/2502.14802v2>
- He, J., Neubig, G., and Berg-Kirkpatrick, T. *Efficient Nearest Neighbor Language Models*, 2021. <https://arxiv.org/abs/2109.04212v3>
- Hu, E. J., Shen, Y., Wallis, P., and others. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*, 2022.
- Huang, Y., Liu, Y., Zhao, R., Zhong, X., Yue, X., and Jiang, L. *MemOrb: A Plug-and-Play Verbal-Reinforcement Memory Layer for E-Commerce Customer Service*, 2025. <https://arxiv.org/abs/2509.18713v1>
- Hwang, J., Park, J., Park, H., Kim, D., Park, S., and Ok, J. *Retrieval-Augmented Generation with Estimation of Source Reliability*, 2024. <https://arxiv.org/abs/2410.22954v5>
- Jiang, J., Wu, X., Song, W., Yang, B., Zhou, Y., Ge, H., Lee, H. P., Liang, Y., and Wu, C. *ReliableRAG: Combating Misinformation in Retrieval-Augmented Generation via Reliability-Guided Reasoning Chains*, 2026. <https://arxiv.org/abs/2608.25487v1>
- Jiang, T., Huang, S., Luo, S., and others. *MoRA: High-Rank Updating for Parameter-Efficient Fine-Tuning*, 2024. <https://arxiv.org/abs/2405.12130>
- Jin, M., Luo, W., Cheng, S., Wang, X., Hua, W., Tang, R., Wang, W. Y., and Zhang, Y. *Disentangling Memory and Reasoning Ability in Large Language Models*, 2024. <https://arxiv.org/abs/2411.13504v3>
- Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. Generalization through Memorization: Nearest Neighbor Language Models. *International Conference on Learning Representations*, 2020. <https://papers.lunadong.com/paper/4555>
- Kim, H., Kim, Y. J., and Bak, J. *PEMA: An Offsite-Tunable Plug-in External Memory Adaptation for Language Models*, 2023. <https://arxiv.org/abs/2311.08590v3>
- Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. *Large Memory Layers with Product Keys*, 2019. <https://arxiv.org/abs/1907.05242v2>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172v3>
- Lyu, Q., Shridhar, K., Malaviya, C., Zhang, L., Elazar, Y., Tandon, N., Apidianaki, M., Sachan, M., and Callison-Burch, C. *Calibrating Large Language Models with Sample Consistency*, 2024. <https://arxiv.org/abs/2402.13904v2>
- Miao, M. M., and Ungar, L. *Closing the Confidence-Faithfulness Gap in Large Language Models*, 2026. <https://arxiv.org/abs/2603.25052v2>
- Nelson, E., Kollias, G., Das, P., Chaudhury, S., and Dan, S. *Needle in the Haystack for Memory Based Large Language Models*, 2024. <https://arxiv.org/abs/2407.01437v2>
- OLAResearch. *Cross-Model Memory Transfer via Target-Side Reader Adaptation*, 2026. <https://github.com/OLAResearch/XMemTransfer>
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. *MemGPT: Towards LLMs as Operating Systems*, 2023. <https://arxiv.org/abs/2310.08560v2>
- Pan, X., Yao, W., Zhang, H., Yu, D., Yu, D., and Chen, J. *Knowledge-in-Context: Towards Knowledgeable Semi-Parametric Language Models*, 2022. <https://arxiv.org/abs/2210.16433v3>
- PioneerQyw. *NGM: A Plug-and-Play Training-Free Memory Module for LLMs*, 2026. <https://github.com/PioneerQyw/NGM>
- Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, 2025. <https://arxiv.org/abs/2501.13956v1>
- Shin, S., Cha, M., Lee, B.-J., and Park, S. *ERA: Evidence-based Reliability Alignment for Honest Retrieval-Augmented Generation*, 2026. <https://arxiv.org/abs/2604.20854v1>
- Wang, H., Luo, Y., and others. *MemTrapBench: Benchmarking Cognitive Traps in LLM Memory Use*, 2026. <https://www.semanticscholar.org/paper/MemTrapBench%3A-Benchmarking-Cognitive-Traps-in-LLM-Wang-Luo/736d61825a5afed4c85b227951a9880d01e2299f>
- Xu, F. F., Alon, U., and Neubig, G. *Why do Nearest Neighbor Language Models Work?*, 2023. <https://arxiv.org/abs/2301.02828v2>
- Xu, J., Shi, H., Shan, Y., Liu, P., Bai, Y., Li, N., and Liu, X. *TrustMargin: Training-Free Arbitration between Para-*

- metric Memory and Retrieved Evidence in Large Language Models*, 2026. <https://arxiv.org/abs/2606.08397v1>
- Yu, Z., Zhao, Y., Cohan, A., and Zhang, X.-P. *HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation*, 2024. <https://arxiv.org/abs/2412.21199v2>
- Yu, Z., Xing, W., Wei, Y., Yang, B., Ye, C., Li, G., and Han, M. *The Attribution Blind Spot: Detecting When Language Models Rely on Memory Rather Than Retrieved Context*, 2026. <https://arxiv.org/abs/2605.26778v1>
- Zhang, Z., Zeng, Z., Lin, Y., Wang, H., Ye, D., Xiao, C., Han, X., Liu, Z., Li, P., Sun, M., and Zhou, J. *Plug-and-Play Knowledge Injection for Pre-trained Language Models*, 2023. <https://arxiv.org/abs/2305.17691v2>
- Zheng, Y., Wen, B., and Wang, X. *Lngram v2: Latent N-Gram Memory with Interpretable Discrete Representations*, 2026a. <https://arxiv.org/abs/2609.03426v1>
- Zheng, Y., Xia, G., Wang, X., and Ren, L. *Lngram: N-gram Conditional Memory in Latent Space*, 2026b. <https://arxiv.org/abs/2605.24869v1>
- Zhong, Z., Lei, T., and Chen, D. *Training Language Models with Memory Augmentation*, 2022. <https://arxiv.org/abs/2205.12674v3>
- Zhou, W., Gu, Y., Iacovides, G., Qiu, Y., Zhao, Q., and Mandic, D. *Tensorizing Engram: Sharing Latents Across N-Gram Embeddings is Beneficial in LLMs*, 2026. <https://arxiv.org/abs/2606.08347v1>

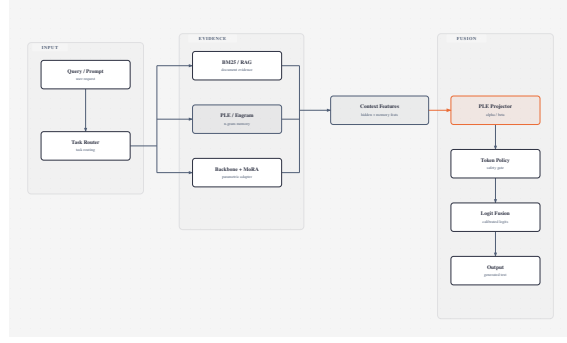


Figure 4. System architecture. The frozen backbone, RAG, and PLE memory provide three complementary evidence channels; the PLE Projector and token policy control logit-level fusion.

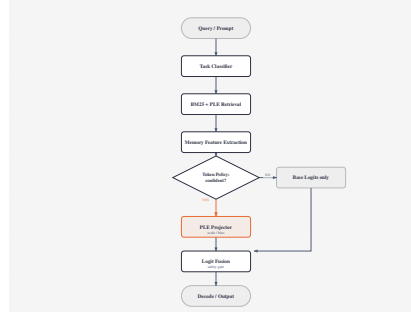


Figure 5. Inference pipeline. The token policy decides whether to apply the learned PLE Projector or bypass it with base logits only.

A. System Architecture and Inference Pipeline

B. Algorithm: PLE Projector Inference

The projector produces a per-token logit correction without modifying the frozen backbone. The learned variables are α_t (memory trust) and β_t (support bias).

Algorithm 1 PLE Projector inference

1. Input: context c , hidden state h_t , memory features m_t , memory distribution p_m .
 2. Compute $\alpha_t, \beta_t = f_{\theta}(h_t, m_t)$.
 3. Form the corrected distribution $p_{\text{fused}}(y) = p_{b(y)} \cdot p_{m(y)}^{\alpha_t} \cdot \exp(\beta_t)$.
 4. Evaluate the token policy $g_t \in \{0, 1\}$.
 5. If $g_t = 0$, reset $\alpha_t \leftarrow 0$ and $\beta_t \leftarrow 0$.
 6. Normalize and return p_{fused} .
-

C. Algorithm: Token-Level Policy and Safety Gate

The policy is a small binary classifier trained on the same memory features used by the projector. It decides whether the n-gram memory should contribute to the current token.

Table 2. 10k local-continuation results, five seeds. NLL is lower-is-better.

Seed	Base NLL	Fixed NLL	Proj. NLL	Base hit	Fixed hit	Proj. hit
0	3.148	3.051	2.930	0.407	0.427	0.463
1	3.328	3.346	3.114	0.410	0.420	0.460
2	3.451	3.426	3.002	0.413	0.430	0.493
3	3.238	3.066	2.848	0.397	0.420	0.473
4	3.197	3.041	2.913	0.400	0.427	0.417

Table 3. kNN-LM and NGM baselines. Neither matches the learned projector.

Baseline	NLL	Hit	Note
Base	2.633	0.505	kNN eval set
kNN-LM	2.847	0.540	better hit, worse NLL
Base	2.521	0.550	NGM eval set
NGM	2.521	0.550	near-zero change

Algorithm 2 Token-level policy and safety gate

1. Train a logistic classifier on paired real/control observations.
 2. At inference, compute features m_t from the current context.
 3. Predict $g_t = P(\text{helpful} \mid m_t)$.
 4. If $g_t < \tau$, disable PLE fusion and use base logits only.
 5. Otherwise, apply the learned projector correction and continue decoding.
-

D. Real-vs-Control Protocol

To distinguish genuine memory use from model noise, we use a strict paired protocol.

1. The real memory is built from documents containing the target continuation.
2. The control memory is built from an unrelated document set with no target relation.
3. A method is considered useful only if it outperforms the control memory by a meaningful margin.
4. All projector results in the paper are paired across identical evaluation rows.

E. Projector Scaling, Baselines and Sensitivity

Per-seed 10k local-continuation results, the kNN-LM and NGM comparisons, and the sweep over n-gram order and memory-bank size.

F. HumanEval and Judge Detail

Under greedy decoding the base model passes HumanEval/16, 18, 23, 25, 32, 33, 38, 41, 43, 46 and 49; BM25+PLE passes HumanEval/0, 10, 27, 35 and 41. The only overlap is HumanEval/41, so BM25+PLE reaches four problems the base model never solves — but it loses nine, and with no confidence interval over 50 problems we treat the recovery as suggestive rather than established.

G. Case Studies

What a Token-Keyed Residual Memory Can Carry

Table 4. Sensitivity over n-gram order and memory-bank size on code continuation. The calibrated fused gain is the NLL improvement of calibrated real fusion over the base model.

Setting	Real-vs-control gap	Calibrated fused gain
order 2	20.23	0.456
order 3	20.69	0.501
order 4	20.49	0.574
order 5	20.53	0.605
memory 40/80	22.68	1.408
memory 120/240	19.47	0.755
memory 300/600	20.28	0.210

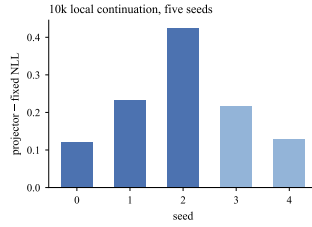


Figure 6. Per-seed projector-versus-fixed improvements on 10k data. Darker bars are seeds whose per-seed artifacts are retained in the released bundle and reproduce Table 2 exactly; the two lighter bars are reported from the original run, whose per-seed files were not retained.

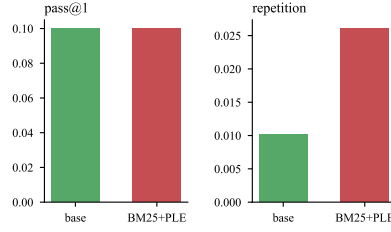


Figure 7. HumanEval pass@1 and mean repetition rate over 50 problems.

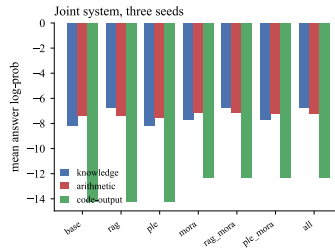


Figure 8. Joint system answer log-probability by resource, three seeds.

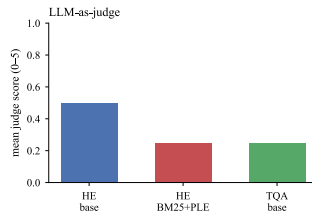


Figure 9. LLM-as-judge mean scores. Judge scores carry run-to-run variance and are reported as secondary evidence.

Table 5. Error analysis and mitigating mechanisms.

Failure mode	Observed evidence	Mitigation
Over-confident n-gram prior	Open-ended repetition and repository fragments	Learned token policy
Missing world knowledge	TriviaQA exact match = 0.005	RAG and parametric adapters
Hidden-state misalignment	Real-vs-control gains near zero	Logit-level calibrated fusion

TriviaQA failure. The frozen 0.8B model obtains 0.005 exact match on the 200-example validation subset. The failure is systematic rather than a calibration artifact: short-form knowledge questions require world knowledge that is not present in local n-gram continuations, and raw PLE fusion cannot recover it.

Open-ended degradation. Without the token-level policy, unconditional PLE fusion on natural-language code prompts produces repetitive fragments, unrelated repository text, and broken structure. The learned policy suppresses fusion on tokens where the n-gram prior is not clearly beneficial, which substantially reduces this degradation.